

Projeto e Análise de Algoritmos

Programação dinâmica

Atílio G. Luiz

Primeiro Semestre de 2026

Programação dinâmica

Problemas e subproblemas

Alguns problemas possuem essas duas propriedades:

- ▶ **subestrutura ótima**

- ▶ decompomos a instância em vários **subproblemas**
- ▶ a solução ótima é construída a partir de soluções ótimas de subproblemas

- ▶ **sobreposição de subproblemas**

- ▶ uma instância é quebrada em várias instâncias menores
- ▶ a recursão recalcula uma mesma instância **várias vezes**

Problemas e subproblemas

Alguns problemas possuem essas duas propriedades:

- ▶ **subestrutura ótima**

- ▶ decompomos a instância em vários **subproblemas**
- ▶ a solução ótima é construída a partir de soluções ótimas de subproblemas

- ▶ **sobreposição de subproblemas**

- ▶ uma instância é quebrada em várias instâncias menores
- ▶ a recursão recalcula uma mesma instância **várias vezes**

Problemas e subproblemas

Alguns problemas possuem essas duas propriedades:

- ▶ **subestrutura ótima**

- ▶ decompomos a instância em vários **subproblemas**
- ▶ a solução ótima é construída a partir de soluções ótimas de subproblemas

- ▶ **sobreposição de subproblemas**

- ▶ uma instância é quebrada em várias instâncias menores
- ▶ a recursão recalcula uma mesma instância **várias vezes**

Problemas e subproblemas

Alguns problemas possuem essas duas propriedades:

- ▶ **subestrutura ótima**

- ▶ decompomos a instância em vários **subproblemas**
- ▶ a solução ótima é construída a partir de soluções ótimas de subproblemas

- ▶ **sobreposição de subproblemas**

- ▶ uma instância é quebrada em várias instâncias menores
- ▶ a recursão recalcula uma mesma instância **várias vezes**

Problemas e subproblemas

Alguns problemas possuem essas duas propriedades:

- ▶ **subestrutura ótima**

- ▶ decompomos a instância em vários **subproblemas**
- ▶ a solução ótima é construída a partir de soluções ótimas de subproblemas

- ▶ **sobreposição de subproblemas**

- ▶ uma instância é quebrada em várias instâncias menores
- ▶ a recursão recalcula uma mesma instância **várias vezes**

Problemas e subproblemas

Alguns problemas possuem essas duas propriedades:

- ▶ **subestrutura ótima**

- ▶ decompomos a instância em vários **subproblemas**
- ▶ a solução ótima é construída a partir de soluções ótimas de subproblemas

- ▶ **sobreposição de subproblemas**

- ▶ uma instância é quebrada em várias instâncias menores
- ▶ a recursão recalcula uma mesma instância **várias vezes**

Problemas e subproblemas

Alguns problemas possuem essas duas propriedades:

- ▶ **subestrutura ótima**

- ▶ decompomos a instância em vários **subproblemas**
- ▶ a solução ótima é construída a partir de soluções ótimas de subproblemas

- ▶ **sobreposição de subproblemas**

- ▶ uma instância é quebrada em várias instâncias menores
- ▶ a recursão recalcula uma mesma instância **várias vezes**

Programação dinâmica

Ideia:

- ▶ evitar o recálculo desnecessário de subproblemas
- ▶ guardar as soluções de subproblemas em uma **tabela**
- ▶ cada entrada da tabela é uma instância distinta do subproblema

Premissas da programação dinâmica:

- ▶ o número de entradas da tabela é pequeno (um polinômio)
- ▶ sabemos computar cada entrada da tabela eficientemente

Programação dinâmica

Ideia:

- ▶ evitar o recálculo desnecessário de subproblemas
- ▶ guardar as soluções de subproblemas em uma **tabela**
- ▶ cada entrada da tabela é uma instância distinta do subproblema

Premissas da programação dinâmica:

- ▶ o número de entradas da tabela é pequeno (um polinômio)
- ▶ sabemos computar cada entrada da tabela eficientemente

Programação dinâmica

Ideia:

- ▶ evitar o recálculo desnecessário de subproblemas
- ▶ guardar as soluções de subproblemas em uma **tabela**
- ▶ cada entrada da tabela é uma instância distinta do subproblema

Premissas da programação dinâmica:

- ▶ o número de entradas da tabela é pequeno (um polinômio)
- ▶ sabemos computar cada entrada da tabela eficientemente

Programação dinâmica

Ideia:

- ▶ evitar o recálculo desnecessário de subproblemas
- ▶ guardar as soluções de subproblemas em uma **tabela**
- ▶ cada entrada da tabela é uma instância distinta do subproblema

Premissas da programação dinâmica:

- ▶ o número de entradas da tabela é pequeno (um polinômio)
- ▶ sabemos computar cada entrada da tabela eficientemente

Programação dinâmica

Ideia:

- ▶ evitar o recálculo desnecessário de subproblemas
- ▶ guardar as soluções de subproblemas em uma **tabela**
- ▶ cada entrada da tabela é uma instância distinta do subproblema

Premissas da programação dinâmica:

- ▶ o número de entradas da tabela é pequeno (um polinômio)
- ▶ sabemos computar cada entrada da tabela eficientemente

Programação dinâmica

Ideia:

- ▶ evitar o recálculo desnecessário de subproblemas
- ▶ guardar as soluções de subproblemas em uma **tabela**
- ▶ cada entrada da tabela é uma instância distinta do subproblema

Premissas da programação dinâmica:

- ▶ o número de entradas da tabela é pequeno (um polinômio)
- ▶ sabemos computar cada entrada da tabela eficientemente

Programação dinâmica

Ideia:

- ▶ evitar o recálculo desnecessário de subproblemas
- ▶ guardar as soluções de subproblemas em uma **tabela**
- ▶ cada entrada da tabela é uma instância distinta do subproblema

Premissas da programação dinâmica:

- ▶ o número de entradas da tabela é pequeno (um polinômio)
- ▶ sabemos computar cada entrada da tabela eficientemente

Problemas de otimização

Consideramos tipicamente **problemas de otimização**

- ▶ cada instância tem um conjunto de **soluções viáveis**
- ▶ cada uma delas tem um valor dado pela **função-objetivo**
- ▶ queremos alguma cujo valor é **mínimo ou máximo**

Uma solução viável com o melhor valor é chamada de **ótima**

Problemas de otimização

Consideramos tipicamente **problemas de otimização**

- ▶ cada instância tem um conjunto de **soluções viáveis**
- ▶ cada uma delas tem um valor dado pela **função-objetivo**
- ▶ queremos alguma cujo valor é **mínimo ou máximo**

Uma solução viável com o melhor valor é chamada de **ótima**

Problemas de otimização

Consideramos tipicamente **problemas de otimização**

- ▶ cada instância tem um conjunto de **soluções viáveis**
- ▶ cada uma delas tem um valor dado pela **função-objetivo**
- ▶ queremos alguma cujo valor é **mínimo ou máximo**

Uma solução viável com o melhor valor é chamada de **ótima**

Problemas de otimização

Consideramos tipicamente **problemas de otimização**

- ▶ cada instância tem um conjunto de **soluções viáveis**
- ▶ cada uma delas tem um valor dado pela **função-objetivo**
- ▶ queremos alguma cujo valor é **mínimo ou máximo**

Uma solução viável com o melhor valor é chamada de **ótima**

Problemas de otimização

Consideramos tipicamente **problemas de otimização**

- ▶ cada instância tem um conjunto de **soluções viáveis**
- ▶ cada uma delas tem um valor dado pela **função-objetivo**
- ▶ queremos alguma cujo valor é **mínimo ou máximo**

Uma solução viável com o melhor valor é chamada de **ótima**

Programação dinâmica

- ▶ Problema do Corte

Corte de barras de aço

- ▶ Uma empresa corta longas barras de aço em pedaços menores para revenda
- ▶ Realizar um corte tem um custo insignificante
- ▶ Uma barra tem tamanho inteiro n e podemos cortá-la em qualquer posição $1 \leq i \leq n-1$ ou revender a barra inteira
- ▶ Um pedaço de tamanho $1 \leq i \leq n$ tem preço de revenda p_i

Corte de barras de aço

- ▶ Uma empresa corta longas barras de aço em pedaços menores para revenda
- ▶ Realizar um corte tem um custo insignificante
- ▶ Uma barra tem tamanho inteiro n e podemos cortá-la em qualquer posição $1 \leq i \leq n-1$ ou revender a barra inteira
- ▶ Um pedaço de tamanho $1 \leq i \leq n$ tem preço de revenda p_i

Corte de barras de aço

- ▶ Uma empresa corta longas barras de aço em pedaços menores para revenda
- ▶ Realizar um corte tem um custo insignificante
- ▶ Uma barra tem tamanho inteiro n e podemos cortá-la em qualquer posição $1 \leq i \leq n-1$ ou revender a barra inteira
- ▶ Um pedaço de tamanho $1 \leq i \leq n$ tem preço de revenda p_i

Corte de barras de aço

- ▶ Uma empresa corta longas barras de aço em pedaços menores para revenda
- ▶ Realizar um corte tem um custo insignificante
- ▶ Uma barra tem tamanho inteiro n e podemos cortá-la em qualquer posição $1 \leq i \leq n-1$ ou revender a barra inteira
- ▶ Um pedaço de tamanho $1 \leq i \leq n$ tem preço de revenda p_i

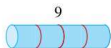
Exemplo de barra de aço

Exemplo onde $n = 4$

tamanho	1	2	3	4
preço	1	5	8	9

Algumas formas de cortar:

- ▶ 4 itens de tamanho 1, preço total 4
- ▶ 2 itens de tamanho 2, preço total 10
- ▶ 2 itens de tamanho 1 e 1 item de tamanho 2, preço total 7



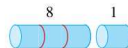
(a)



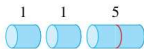
(b)



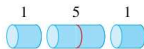
(c)



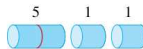
(d)



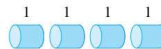
(e)



(f)



(g)



(h)

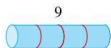
Exemplo de barra de aço

Exemplo onde $n = 4$

tamanho	1	2	3	4
preço	1	5	8	9

Algumas formas de cortar:

- ▶ 4 itens de tamanho 1, preço total 4
- ▶ 2 itens de tamanho 2, preço total 10
- ▶ 2 itens de tamanho 1 e 1 item de tamanho 2, preço total 7



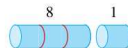
(a)



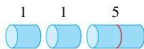
(b)



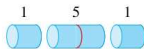
(c)



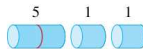
(d)



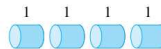
(e)



(f)



(g)



(h)

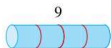
Exemplo de barra de aço

Exemplo onde $n = 4$

tamanho	1	2	3	4
preço	1	5	8	9

Algumas formas de cortar:

- ▶ 4 itens de tamanho 1, preço total 4
- ▶ 2 itens de tamanho 2, preço total 10
- ▶ 2 itens de tamanho 1 e 1 item de tamanho 2, preço total 7



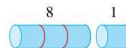
(a)



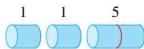
(b)



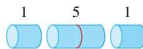
(c)



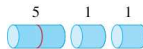
(d)



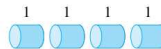
(e)



(f)



(g)



(h)

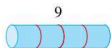
Exemplo de barra de aço

Exemplo onde $n = 4$

tamanho	1	2	3	4
preço	1	5	8	9

Algumas formas de cortar:

- ▶ 4 itens de tamanho 1, **preço total 4**
- ▶ 2 itens de tamanho 2, **preço total 10**
- ▶ 2 itens de tamanho 1 e 1 item de tamanho 2, **preço total 7**



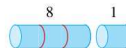
(a)



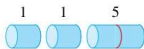
(b)



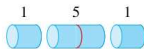
(c)



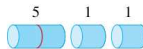
(d)



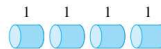
(e)



(f)



(g)



(h)

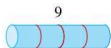
Exemplo de barra de aço

Exemplo onde $n = 4$

tamanho	1	2	3	4
preço	1	5	8	9

Algumas formas de cortar:

- ▶ 4 itens de tamanho 1, preço total 4
- ▶ 2 itens de tamanho 2, preço total 10
- ▶ 2 itens de tamanho 1 e 1 item de tamanho 2, preço total 7



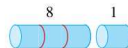
(a)



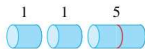
(b)



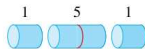
(c)



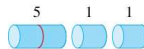
(d)



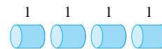
(e)



(f)



(g)



(h)

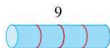
Exemplo de barra de aço

Exemplo onde $n = 4$

tamanho	1	2	3	4
preço	1	5	8	9

Algumas formas de cortar:

- ▶ 4 itens de tamanho 1, preço total 4
- ▶ 2 itens de tamanho 2, preço total 10
- ▶ 2 itens de tamanho 1 e 1 item de tamanho 2, preço total 7



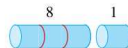
(a)



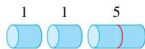
(b)



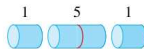
(c)



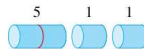
(d)



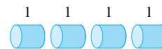
(e)



(f)



(g)



(h)

Problema do corte de barra de aço

Problema:

- ▶ Entrada:
 - ▶ inteiro n
 - ▶ vetor p com os preços de revenda
- ▶ Solução:
 - ▶ valor do preço máximo de revenda que pode ser obtido e quais cortes realizar na barra
- ▶ Objetivo:
 - ▶ maximizar preço total de revenda dos pedaços

Problema do corte de barra de aço

Problema:

- ▶ Entrada:
 - ▶ inteiro n
 - ▶ vetor p com os preços de revenda
- ▶ Solução:
 - ▶ valor do preço máximo de revenda que pode ser obtido e quais cortes realizar na barra
- ▶ Objetivo:
 - ▶ maximizar preço total de revenda dos pedaços

Problema do corte de barra de aço

Problema:

- ▶ Entrada:
 - ▶ inteiro n
 - ▶ vetor p com os preços de revenda
- ▶ Solução:
 - ▶ valor do preço máximo de revenda que pode ser obtido e quais cortes realizar na barra
- ▶ Objetivo:
 - ▶ maximizar preço total de revenda dos pedaços

Problema do corte de barra de aço

Problema:

- ▶ Entrada:
 - ▶ inteiro n
 - ▶ vetor p com os preços de revenda
- ▶ Solução:
 - ▶ valor do preço máximo de revenda que pode ser obtido e quais cortes realizar na barra
- ▶ Objetivo:
 - ▶ maximizar preço total de revenda dos pedaços

Problema do corte de barra de aço

Problema:

- ▶ Entrada:
 - ▶ inteiro n
 - ▶ vetor p com os preços de revenda
- ▶ Solução:
 - ▶ valor do preço máximo de revenda que pode ser obtido e quais cortes realizar na barra
- ▶ Objetivo:
 - ▶ maximizar preço total de revenda dos pedaços

Problema do corte de barra de aço

Problema:

- ▶ Entrada:
 - ▶ inteiro n
 - ▶ vetor p com os preços de revenda
- ▶ Solução:
 - ▶ valor do preço máximo de revenda que pode ser obtido e quais cortes realizar na barra
- ▶ Objetivo:
 - ▶ maximizar preço total de revenda dos pedaços

Problema do corte de barra de aço

Problema:

- ▶ Entrada:
 - ▶ inteiro n
 - ▶ vetor p com os preços de revenda
- ▶ Solução:
 - ▶ valor do preço máximo de revenda que pode ser obtido e quais cortes realizar na barra
- ▶ Objetivo:
 - ▶ maximizar preço total de revenda dos pedaços

Problema do corte de barra de aço

Problema:

- ▶ Entrada:
 - ▶ inteiro n
 - ▶ vetor p com os preços de revenda
- ▶ Solução:
 - ▶ valor do preço máximo de revenda que pode ser obtido e quais cortes realizar na barra
- ▶ Objetivo:
 - ▶ maximizar preço total de revenda dos pedaços

Solução por força bruta

- ▶ Há 2^{n-1} modos diferentes de cortar uma barra de aço de tamanho n .
- ▶ Logo, um algoritmo força bruta torna-se impraticável para valores grandes de n .

Solução por força bruta

- ▶ Há 2^{n-1} modos diferentes de cortar uma barra de aço de tamanho n .
- ▶ Logo, um algoritmo força bruta torna-se impraticável para valores grandes de n .

Projeto por indução

Seja r_n o preço total ótimo para uma barra de tamanho n

► Base:

► se $n = 0$ então $r_0 = 0$

► Hipótese de indução:

► sabemos calcular r_k para $k < n$

► Passo:

► seja i o tamanho do primeiro pedaço da solução ótima

► a melhor forma de cortar o resto da barra vale r_{n-i} , daí

$$r_n = p_i + r_{n-i}$$

► como não conhecemos o tamanho do primeiro pedaço,

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

Projeto por indução

Seja r_n o preço total ótimo para uma barra de tamanho n

- ▶ Base:

- ▶ se $n = 0$ então $r_0 = 0$

- ▶ Hipótese de indução:

- ▶ sabemos calcular r_k para $k < n$

- ▶ Passo:

- ▶ seja i o tamanho do primeiro pedaço da solução ótima

- ▶ a melhor forma de cortar o resto da barra vale r_{n-i} , daí

$$r_n = p_i + r_{n-i}$$

- ▶ como não conhecemos o tamanho do primeiro pedaço,

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

Projeto por indução

Seja r_n o preço total ótimo para uma barra de tamanho n

- ▶ Base:

- ▶ se $n = 0$ então $r_0 = 0$

- ▶ Hipótese de indução:

- ▶ sabemos calcular r_k para $k < n$

- ▶ Passo:

- ▶ seja i o tamanho do primeiro pedaço da solução ótima

- ▶ a melhor forma de cortar o resto da barra vale r_{n-i} , daí

$$r_n = p_i + r_{n-i}$$

- ▶ como não conhecemos o tamanho do primeiro pedaço,

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

Projeto por indução

Seja r_n o preço total ótimo para uma barra de tamanho n

- ▶ Base:

- ▶ se $n = 0$ então $r_0 = 0$

- ▶ Hipótese de indução:

- ▶ sabemos calcular r_k para $k < n$

- ▶ Passo:

- ▶ seja i o tamanho do primeiro pedaço da solução ótima

- ▶ a melhor forma de cortar o resto da barra vale r_{n-i} , daí

$$r_n = p_i + r_{n-i}$$

- ▶ como não conhecemos o tamanho do primeiro pedaço,

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

Projeto por indução

Seja r_n o preço total ótimo para uma barra de tamanho n

► Base:

► se $n = 0$ então $r_0 = 0$

► Hipótese de indução:

► sabemos calcular r_k para $k < n$

► Passo:

► seja i o tamanho do primeiro pedaço da solução ótima

► a melhor forma de cortar o resto da barra vale r_{n-i} , daí

$$r_n = p_i + r_{n-i}$$

► como não conhecemos o tamanho do primeiro pedaço,

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

Projeto por indução

Seja r_n o preço total ótimo para uma barra de tamanho n

- ▶ Base:
 - ▶ se $n = 0$ então $r_0 = 0$
- ▶ Hipótese de indução:
 - ▶ sabemos calcular r_k para $k < n$
- ▶ Passo:
 - ▶ seja i o tamanho do primeiro pedaço da solução ótima
 - ▶ a melhor forma de cortar o resto da barra vale r_{n-i} , daí

$$r_n = p_i + r_{n-i}$$

- ▶ como não conhecemos o tamanho do primeiro pedaço,

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

Projeto por indução

Seja r_n o preço total ótimo para uma barra de tamanho n

- ▶ Base:
 - ▶ se $n = 0$ então $r_0 = 0$
- ▶ Hipótese de indução:
 - ▶ sabemos calcular r_k para $k < n$
- ▶ Passo:
 - ▶ seja i o tamanho do primeiro pedaço da solução ótima
 - ▶ a melhor forma de cortar o resto da barra vale r_{n-i} , daí

$$r_n = p_i + r_{n-i}$$

- ▶ como não conhecemos o tamanho do primeiro pedaço,

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

Projeto por indução

Seja r_n o preço total ótimo para uma barra de tamanho n

- ▶ Base:
 - ▶ se $n = 0$ então $r_0 = 0$
- ▶ Hipótese de indução:
 - ▶ sabemos calcular r_k para $k < n$
- ▶ Passo:
 - ▶ seja i o tamanho do primeiro pedaço da solução ótima
 - ▶ a melhor forma de cortar o resto da barra vale r_{n-i} , daí

$$r_n = p_i + r_{n-i}$$

- ▶ como não conhecemos o tamanho do primeiro pedaço,

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

Projeto por indução

Seja r_n o preço total ótimo para uma barra de tamanho n

- ▶ Base:
 - ▶ se $n = 0$ então $r_0 = 0$
- ▶ Hipótese de indução:
 - ▶ sabemos calcular r_k para $k < n$
- ▶ Passo:
 - ▶ seja i o tamanho do primeiro pedaço da solução ótima
 - ▶ a melhor forma de cortar o resto da barra vale r_{n-i} , daí

$$r_n = p_i + r_{n-i}$$

- ▶ como não conhecemos o tamanho do primeiro pedaço,

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

Algoritmo recursivo

Cut-Rod(p, n)

```
1  se  $n == 0$  então
2    devolva 0
3   $s = 0$ 
4  para  $i = 1$  até  $n$  faça
5     $t = p[i] + \text{CUT-ROD}(p, n - i)$ 
6    se  $t > s$  então
7       $s = t$ 
8  devolva  $s$ 
```

A complexidade de tempo do algoritmo é dada por

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 0 \\ \sum_{j=0}^{n-1} T(j) + \Theta(1) & \text{se } n > 0 \end{cases}$$

Algoritmo recursivo

```
Cut-Rod( $p, n$ )  
1  se  $n == 0$  então  
2    devolva 0  
3   $s = 0$   
4  para  $i = 1$  até  $n$  faça  
5     $t = p[i] + \text{CUT-ROD}(p, n - i)$   
6    se  $t > s$  então  
7       $s = t$   
8  devolva  $s$ 
```

A complexidade de tempo do algoritmo é dada por

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 0 \\ \sum_{j=0}^{n-1} T(j) + \Theta(1) & \text{se } n > 0 \end{cases}$$

Algoritmo recursivo

```
Cut-Rod( $p, n$ )  
1  se  $n == 0$  então  
2    devolva 0  
3   $s = 0$   
4  para  $i = 1$  até  $n$  faça  
5     $t = p[i] + \text{CUT-ROD}(p, n - i)$   
6    se  $t > s$  então  
7       $s = t$   
8  devolva  $s$ 
```

A complexidade de tempo do algoritmo é dada por

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 0 \\ \sum_{j=0}^{n-1} T(j) + \Theta(1) & \text{se } n > 0 \end{cases}$$

Tempo do algoritmo recursivo

Subtraindo $T(n-1)$ de $T(n)$ obtemos

$$T(n) - T(n-1) = \left(\sum_{j=0}^{n-1} T(j) + c \right) - \left(\sum_{j=0}^{n-2} T(j) + c \right) = T(n-1)$$

Ou seja

$$T(n) = 2T(n-1)$$

Resolvendo esta última recorrência, obtemos

$$T(n) = \Theta(2^n)$$

Tempo do algoritmo recursivo

Subtraindo $T(n-1)$ de $T(n)$ obtemos

$$T(n) - T(n-1) = \left(\sum_{j=0}^{n-1} T(j) + c \right) - \left(\sum_{j=0}^{n-2} T(j) + c \right) = T(n-1)$$

Ou seja

$$T(n) = 2T(n-1)$$

Resolvendo esta última recorrência, obtemos

$$T(n) = \Theta(2^n)$$

Tempo do algoritmo recursivo

Subtraindo $T(n-1)$ de $T(n)$ obtemos

$$T(n) - T(n-1) = \left(\sum_{j=0}^{n-1} T(j) + c \right) - \left(\sum_{j=0}^{n-2} T(j) + c \right) = T(n-1)$$

Ou seja

$$T(n) = 2T(n-1)$$

Resolvendo esta última recorrência, obtemos

$$T(n) = \Theta(2^n)$$

Tempo do algoritmo recursivo

Subtraindo $T(n-1)$ de $T(n)$ obtemos

$$T(n) - T(n-1) = \left(\sum_{j=0}^{n-1} T(j) + c \right) - \left(\sum_{j=0}^{n-2} T(j) + c \right) = T(n-1)$$

Ou seja

$$T(n) = 2T(n-1)$$

Resolvendo esta última recorrência, obtemos

$$T(n) = \Theta(2^n)$$

Tempo do algoritmo recursivo

Subtraindo $T(n-1)$ de $T(n)$ obtemos

$$T(n) - T(n-1) = \left(\sum_{j=0}^{n-1} T(j) + c \right) - \left(\sum_{j=0}^{n-2} T(j) + c \right) = T(n-1)$$

Ou seja

$$T(n) = 2T(n-1)$$

Resolvendo esta última recorrência, obtemos

$$T(n) = \Theta(2^n)$$

Tempo do algoritmo recursivo

Subtraindo $T(n-1)$ de $T(n)$ obtemos

$$T(n) - T(n-1) = \left(\sum_{j=0}^{n-1} T(j) + c \right) - \left(\sum_{j=0}^{n-2} T(j) + c \right) = T(n-1)$$

Ou seja

$$T(n) = 2T(n-1)$$

Resolvendo esta última recorrência, obtemos

$$T(n) = \Theta(2^n)$$

Tempo do algoritmo recursivo

Subtraindo $T(n-1)$ de $T(n)$ obtemos

$$T(n) - T(n-1) = \left(\sum_{j=0}^{n-1} T(j) + c \right) - \left(\sum_{j=0}^{n-2} T(j) + c \right) = T(n-1)$$

Ou seja

$$T(n) = 2T(n-1)$$

Resolvendo esta última recorrência, obtemos

$$T(n) = \Theta(2^n)$$

Tempo do algoritmo recursivo

Subtraindo $T(n-1)$ de $T(n)$ obtemos

$$T(n) - T(n-1) = \left(\sum_{j=0}^{n-1} T(j) + c \right) - \left(\sum_{j=0}^{n-2} T(j) + c \right) = T(n-1)$$

Ou seja

$$T(n) = 2T(n-1)$$

Resolvendo esta última recorrência, obtemos

$$T(n) = \Theta(2^n)$$

Número de subproblemas

- ▶ temos um algoritmo exponencial, que tem pouca utilidade
 - ▶ para uma entrada $n = 40$ ele pode levar até 1 hora para terminar, dependendo da máquina

Número de subproblemas:

- ▶ Note que o parâmetro de entrada n varia apenas de 0 a $n - 1$
- ▶ ou seja, há apenas $O(n)$ instâncias do subproblema

Número de subproblemas

- ▶ temos um algoritmo exponencial, que tem pouca utilidade
 - ▶ para uma entrada $n = 40$ ele pode levar até 1 hora para terminar, dependendo da máquina

Número de subproblemas:

- ▶ Note que o parâmetro de entrada n varia apenas de 0 a $n - 1$
- ▶ ou seja, há apenas $O(n)$ instâncias do subproblema

Número de subproblemas

- ▶ temos um algoritmo exponencial, que tem pouca utilidade
 - ▶ para uma entrada $n = 40$ ele pode levar até 1 hora para terminar, dependendo da máquina

Número de subproblemas:

- ▶ Note que o parâmetro de entrada n varia apenas de 0 a $n - 1$
- ▶ ou seja, há apenas $O(n)$ instâncias do subproblema

Número de subproblemas

- ▶ temos um algoritmo exponencial, que tem pouca utilidade
 - ▶ para uma entrada $n = 40$ ele pode levar até 1 hora para terminar, dependendo da máquina

Número de subproblemas:

- ▶ Note que o parâmetro de entrada n varia apenas de 0 a $n - 1$
- ▶ ou seja, há apenas $O(n)$ instâncias do subproblema

Número de subproblemas

- ▶ temos um algoritmo exponencial, que tem pouca utilidade
 - ▶ para uma entrada $n = 40$ ele pode levar até 1 hora para terminar, dependendo da máquina

Número de subproblemas:

- ▶ Note que o parâmetro de entrada n varia apenas de 0 a $n - 1$
- ▶ ou seja, há apenas $O(n)$ instâncias do subproblema

Número de subproblemas

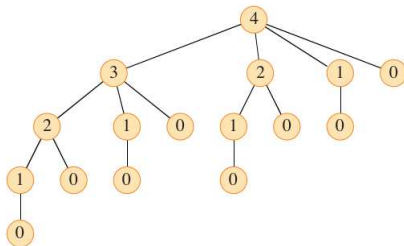


Figura: Árvore de recursão das chamadas recursivas de $\text{CUT-ROD}(p, 4)$. Um caminho da raiz até uma folha corresponde a uma das 2^{n-1} formas de cortar uma barra de aço de tamanho $n = 4$.

- ▶ vários subproblemas aparecem repetidas vezes
- ▶ Como fazer para não recomputar o que já foi computado antes?

Número de subproblemas

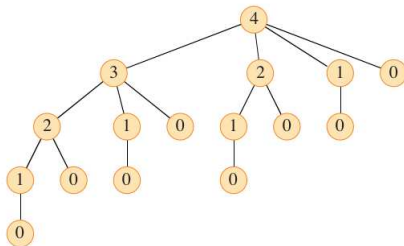


Figura: Árvore de recursão das chamadas recursivas de $\text{CUT-ROD}(p, 4)$. Um caminho da raiz até uma folha corresponde a uma das 2^{n-1} formas de cortar uma barra de aço de tamanho $n = 4$.

- ▶ vários subproblemas aparecem repetidas vezes
- ▶ Como fazer para não recomputar o que já foi computado antes?

Duas estratégias

1. Top-down

- ▶ resolvemos os subproblemas novos recursivamente
- ▶ guardamos os resultados em uma estrutura de dados
- ▶ é chamada de programação dinâmica com memoização ou simplesmente de **memoização**.

2. Bottom-up

- ▶ criamos uma tabela para guardar os resultados
- ▶ preenchemos as entradas para os casos básicos
- ▶ preenchemos as demais, das menores para as maiores
- ▶ é chamada tipicamente de **programação dinâmica**

Duas estratégias

1. Top-down

- ▶ resolvemos os subproblemas novos recursivamente
- ▶ guardamos os resultados em uma estrutura de dados
- ▶ é chamada de programação dinâmica com memoização ou simplesmente de **memoização**.

2. Bottom-up

- ▶ criamos uma tabela para guardar os resultados
- ▶ preenchemos as entradas para os casos básicos
- ▶ preenchemos as demais, das menores para as maiores
- ▶ é chamada tipicamente de **programação dinâmica**

Duas estratégias

1. Top-down

- ▶ resolvemos os subproblemas novos recursivamente
- ▶ guardamos os resultados em uma estrutura de dados
- ▶ é chamada de programação dinâmica com memoização ou simplesmente de **memoização**.

2. Bottom-up

- ▶ criamos uma tabela para guardar os resultados
- ▶ preenchemos as entradas para os casos básicos
- ▶ preenchemos as demais, das menores para as maiores
- ▶ é chamada tipicamente de **programação dinâmica**

Duas estratégias

1. Top-down

- ▶ resolvemos os subproblemas novos recursivamente
- ▶ guardamos os resultados em uma estrutura de dados
- ▶ é chamada de programação dinâmica com memoização ou simplesmente de **memoização**.

2. Bottom-up

- ▶ criamos uma tabela para guardar os resultados
- ▶ preenchemos as entradas para os casos básicos
- ▶ preenchemos as demais, das menores para as maiores
- ▶ é chamada tipicamente de **programação dinâmica**

Duas estratégias

1. Top-down

- ▶ resolvemos os subproblemas novos recursivamente
- ▶ guardamos os resultados em uma estrutura de dados
- ▶ é chamada de programação dinâmica com memoização ou simplesmente de **memoização**.

2. Bottom-up

- ▶ criamos uma tabela para guardar os resultados
- ▶ preenchemos as entradas para os casos básicos
- ▶ preenchemos as demais, das menores para as maiores
- ▶ é chamada tipicamente de **programação dinâmica**

Duas estratégias

1. Top-down

- ▶ resolvemos os subproblemas novos recursivamente
- ▶ guardamos os resultados em uma estrutura de dados
- ▶ é chamada de programação dinâmica com memoização ou simplesmente de **memoização**.

2. Bottom-up

- ▶ criamos uma tabela para guardar os resultados
- ▶ preenchemos as entradas para os casos básicos
- ▶ preenchemos as demais, das menores para as maiores
- ▶ é chamada tipicamente de **programação dinâmica**

Duas estratégias

1. Top-down

- ▶ resolvemos os subproblemas novos recursivamente
- ▶ guardamos os resultados em uma estrutura de dados
- ▶ é chamada de programação dinâmica com memoização ou simplesmente de **memoização**.

2. Bottom-up

- ▶ criamos uma tabela para guardar os resultados
- ▶ preenchemos as entradas para os casos básicos
- ▶ preenchemos as demais, das menores para as maiores
- ▶ é chamada tipicamente de **programação dinâmica**

Duas estratégias

1. Top-down

- ▶ resolvemos os subproblemas novos recursivamente
- ▶ guardamos os resultados em uma estrutura de dados
- ▶ é chamada de programação dinâmica com memoização ou simplesmente de **memoização**.

2. Bottom-up

- ▶ criamos uma tabela para guardar os resultados
- ▶ preenchemos as entradas para os casos básicos
- ▶ preenchemos as demais, das menores para as maiores
- ▶ é chamada tipicamente de **programação dinâmica**

Duas estratégias

1. Top-down

- ▶ resolvemos os subproblemas novos recursivamente
- ▶ guardamos os resultados em uma estrutura de dados
- ▶ é chamada de programação dinâmica com memoização ou simplesmente de **memoização**.

2. Bottom-up

- ▶ criamos uma tabela para guardar os resultados
- ▶ preenchemos as entradas para os casos básicos
- ▶ preenchemos as demais, das menores para as maiores
- ▶ é chamada tipicamente de **programação dinâmica**

Programação Dinâmica com Memoização

- ▶ Após observar que uma solução recursiva é ineficiente porque resolve o mesmo subproblema repetidas vezes, forçamos para que cada subproblema seja resolvido **apenas uma vez**, salvando sua solução numa **tabela**
- ▶ Se precisarmos do valor da solução do subproblema futuramente, é só pesquisar na tabela em tempo $O(1)$.
- ▶ Assim, a programação dinâmica usa memória adicional a fim de ganhar tempo de computação.
 - ▶ em muitos casos, um algoritmo exponencial pode se tornar um algoritmo polinomial
- ▶ Um algoritmo de programação dinâmica executa em tempo polinomial quando:
 1. o número de subproblemas distintos é polinomial no tamanho da entrada e
 2. quando podemos resolver cada subproblema em tempo polinomial.

Programação Dinâmica com Memoização

- ▶ Após observar que uma solução recursiva é ineficiente porque resolve o mesmo subproblema repetidas vezes, forçamos para que cada subproblema seja resolvido **apenas uma vez**, salvando sua solução numa **tabela**
- ▶ Se precisarmos do valor da solução do subproblema futuramente, é só pesquisar na tabela em tempo $O(1)$.
- ▶ Assim, a programação dinâmica usa memória adicional a fim de ganhar tempo de computação.
 - ▶ em muitos casos, um algoritmo exponencial pode se tornar um algoritmo polinomial
- ▶ Um algoritmo de programação dinâmica executa em tempo polinomial quando:
 1. o número de subproblemas distintos é polinomial no tamanho da entrada e
 2. quando podemos resolver cada subproblema em tempo polinomial.

Programação Dinâmica com Memoização

- ▶ Após observar que uma solução recursiva é ineficiente porque resolve o mesmo subproblema repetidas vezes, forçamos para que cada subproblema seja resolvido **apenas uma vez**, salvando sua solução numa **tabela**
- ▶ Se precisarmos do valor da solução do subproblema futuramente, é só pesquisar na tabela em tempo $O(1)$.
- ▶ Assim, a programação dinâmica usa memória adicional a fim de ganhar tempo de computação.
 - ▶ em muitos casos, um algoritmo exponencial pode se tornar um algoritmo polinomial
- ▶ Um algoritmo de programação dinâmica executa em tempo polinomial quando:
 1. o número de subproblemas distintos é polinomial no tamanho da entrada e
 2. quando podemos resolver cada subproblema em tempo polinomial.

Programação Dinâmica com Memoização

- ▶ Após observar que uma solução recursiva é ineficiente porque resolve o mesmo subproblema repetidas vezes, forçamos para que cada subproblema seja resolvido **apenas uma vez**, salvando sua solução numa **tabela**
- ▶ Se precisarmos do valor da solução do subproblema futuramente, é só pesquisar na tabela em tempo $O(1)$.
- ▶ Assim, a programação dinâmica usa memória adicional a fim de ganhar tempo de computação.
 - ▶ em muitos casos, um algoritmo exponencial pode se tornar um algoritmo polinomial
- ▶ Um algoritmo de programação dinâmica executa em tempo polinomial quando:
 1. o número de subproblemas distintos é polinomial no tamanho da entrada e
 2. quando podemos resolver cada subproblema em tempo polinomial.

Programação Dinâmica com Memoização

- ▶ Após observar que uma solução recursiva é ineficiente porque resolve o mesmo subproblema repetidas vezes, forçamos para que cada subproblema seja resolvido **apenas uma vez**, salvando sua solução numa **tabela**
- ▶ Se precisarmos do valor da solução do subproblema futuramente, é só pesquisar na tabela em tempo $O(1)$.
- ▶ Assim, a programação dinâmica usa memória adicional a fim de ganhar tempo de computação.
 - ▶ em muitos casos, um algoritmo exponencial pode se tornar um algoritmo polinomial
- ▶ Um algoritmo de programação dinâmica executa em tempo polinomial quando:
 1. o número de subproblemas distintos é polinomial no tamanho da entrada e
 2. quando podemos resolver cada subproblema em tempo polinomial.

Programação Dinâmica com Memoização

- ▶ Após observar que uma solução recursiva é ineficiente porque resolve o mesmo subproblema repetidas vezes, forçamos para que cada subproblema seja resolvido **apenas uma vez**, salvando sua solução numa **tabela**
- ▶ Se precisarmos do valor da solução do subproblema futuramente, é só pesquisar na tabela em tempo $O(1)$.
- ▶ Assim, a programação dinâmica usa memória adicional a fim de ganhar tempo de computação.
 - ▶ em muitos casos, um algoritmo exponencial pode se tornar um algoritmo polinomial
- ▶ Um algoritmo de programação dinâmica executa em tempo polinomial quando:
 1. o número de subproblemas distintos é polinomial no tamanho da entrada e
 2. quando podemos resolver cada subproblema em tempo polinomial.

Programação Dinâmica com Memoização

- ▶ Após observar que uma solução recursiva é ineficiente porque resolve o mesmo subproblema repetidas vezes, forçamos para que cada subproblema seja resolvido apenas uma vez, salvando sua solução numa tabela
- ▶ Se precisarmos do valor da solução do subproblema futuramente, é só pesquisar na tabela em tempo $O(1)$.
- ▶ Assim, a programação dinâmica usa memória adicional a fim de ganhar tempo de computação.
 - ▶ em muitos casos, um algoritmo exponencial pode se tornar um algoritmo polinomial
- ▶ Um algoritmo de programação dinâmica executa em tempo polinomial quando:
 1. o número de subproblemas distintos é polinomial no tamanho da entrada e
 2. quando podemos resolver cada subproblema em tempo polinomial.

Algoritmo de PD — Estratégia Top-Down

- ▶ Onde guardar as soluções já computadas?
- ▶ Criamos um vetor $r[0...n]$ para usar programação dinâmica

Memoized-Cut-Rod(p, n)

- 1 seja $r[0...n]$ um novo array
- 2 para $i = 0$ até n faça
- 3 $r[i] = -1$
- 4 devolva MEMOIZED-CUT-ROD-AUX(p, n, r)

Algoritmo de PD — Estratégia Top-Down

- ▶ Onde guardar as soluções já computadas?
- ▶ Criamos um vetor $r[0...n]$ para usar programação dinâmica

Memoized-Cut-Rod(p, n)

- 1 seja $r[0...n]$ um novo array
- 2 para $i = 0$ até n faça
- 3 $r[i] = -1$
- 4 devolva MEMOIZED-CUT-ROD-AUX(p, n, r)

Algoritmo de PD — Estratégia Top-Down

- ▶ Onde guardar as soluções já computadas?
- ▶ Criamos um vetor $r[0...n]$ para usar programação dinâmica

Memoized-Cut-Rod (p, n)

- 1 seja $r[0...n]$ um novo array
- 2 para $i = 0$ até n faça
- 3 $r[i] = -1$
- 4 devolva MEMOIZED-CUT-ROD-AUX(p, n, r)

Algoritmo de PD — Top-Down

Memoized-Cut-Rod-Aux(p, n, r)

```
1  se  $r[n] \geq 0$  então
2    devolva  $r[n]$ 
3  se  $n == 0$  então
4     $r[n] = 0$ 
5    devolva 0
6  senão
7     $s = 0$ 
8    para  $i = 1$  até  $n$  faça
9       $q = p[i] + \text{MEMOIZED-CUT-ROD-AUX}(p, n - i, r)$ 
10     se  $q > s$  então
11        $s = q$ 
12    $r[n] = s$ 
13   devolva  $s$ 
```

- O algoritmo acima tem complexidade de tempo $\Theta(n^2)$

Algoritmo de PD — Top-Down

Memoized-Cut-Rod-Aux(p, n, r)

```
1  se  $r[n] \geq 0$  então
2    devolva  $r[n]$ 
3  se  $n == 0$  então
4     $r[n] = 0$ 
5    devolva 0
6  senão
7     $s = 0$ 
8    para  $i = 1$  até  $n$  faça
9       $q = p[i] + \text{MEMOIZED-CUT-ROD-AUX}(p, n - i, r)$ 
10     se  $q > s$  então
11        $s = q$ 
12    $r[n] = s$ 
13   devolva  $s$ 
```

- O algoritmo acima tem complexidade de tempo $\Theta(n^2)$

Grafo de subproblemas

- ▶ O **grafo de subproblemas** é um grafo direcionado contendo um vértice para cada subproblema distinto.
- ▶ Existe um arco de um vértice x para um vértice y se, determinar uma solução para o subproblema x envolve considerar diretamente uma solução ótima do subproblema y .

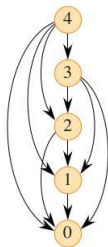
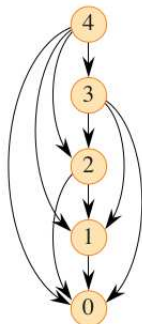


Figura: Grafo de subproblemas para o problema de corte com $n = 4$. Esse grafo é uma versão simplificada da árvore de chamadas recursivas do $\text{CUT-ROD}(p, 4)$

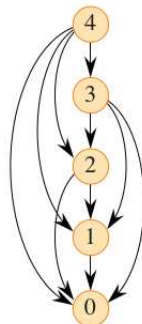
Grafo de subproblemas e complexidade de PD

- ▶ O número de arestas no grafo de subproblemas pode nos ajudar a determinar o tempo de execução de um algoritmo de programação dinâmica.
- ▶ Como resolvemos cada subproblema apenas uma vez, o tempo de execução é a soma do número de vezes necessárias para se resolver cada subproblema.
- ▶ Ou seja, o tempo para computar um subproblema é proporcional ao número de arestas de saída do vértice correspondente no grafo de subproblemas.
- ▶ Por exemplo, no grafo ao lado, existem $\Theta(n^2)$ chamadas recursivas.



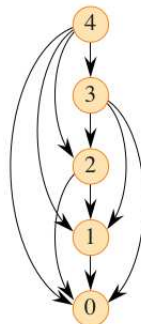
Grafo de subproblemas e complexidade de PD

- ▶ O número de arestas no grafo de subproblemas pode nos ajudar a determinar o tempo de execução de um algoritmo de programação dinâmica.
- ▶ Como resolvemos cada subproblema apenas uma vez, o tempo de execução é a soma do número de vezes necessárias para se resolver cada subproblema.
- ▶ Ou seja, o tempo para computar um subproblema é proporcional ao número de arestas de saída do vértice correspondente no grafo de subproblemas.
- ▶ Por exemplo, no grafo ao lado, existem $\Theta(n^2)$ chamadas recursivas.



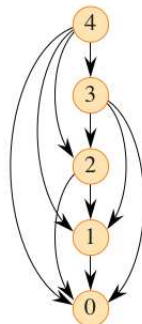
Grafo de subproblemas e complexidade de PD

- ▶ O número de arestas no grafo de subproblemas pode nos ajudar a determinar o tempo de execução de um algoritmo de programação dinâmica.
- ▶ Como resolvemos cada subproblema apenas uma vez, o tempo de execução é a soma do número de vezes necessárias para se resolver cada subproblema.
- ▶ Ou seja, o tempo para computar um subproblema é proporcional ao número de arestas de saída do vértice correspondente no grafo de subproblemas.
- ▶ Por exemplo, no grafo ao lado, existem $\Theta(n^2)$ chamadas recursivas.



Grafo de subproblemas e complexidade de PD

- ▶ O número de arestas no grafo de subproblemas pode nos ajudar a determinar o tempo de execução de um algoritmo de programação dinâmica.
- ▶ Como resolvemos cada subproblema apenas uma vez, o tempo de execução é a soma do número de vezes necessárias para se resolver cada subproblema.
- ▶ Ou seja, o tempo para computar um subproblema é proporcional ao número de arestas de saída do vértice correspondente no grafo de subproblemas.
- ▶ Por exemplo, no grafo ao lado, existem $\Theta(n^2)$ chamadas recursivas.



Algoritmo de PD — Bottom-Up

- ▶ criamos um vetor auxiliar $r[0 \dots n]$ para usar programação dinâmica
- ▶ preenchemos do caso base para os casos maiores

```
BottomUp-Cut-Rod( $p, n$ )  
1  seja  $r[0 \dots n]$  um novo array  
2   $r[0] = 0$   
3  para  $j = 1$  até  $n$  faça  
4       $s = 0$   
5      para  $i = 1$  até  $j$  faça  
6           $t = p[i] + r[j - i]$   
7          se  $t > s$  então  
8               $s = t$   
9       $r[j] = s$   
10 devolva  $r[n]$ 
```

- ▶ o algoritmo tem complexidade de tempo $\Theta(n^2)$

Algoritmo de PD — Bottom-Up

- ▶ criamos um vetor auxiliar $r[0 \dots n]$ para usar programação dinâmica
- ▶ preenchemos do caso base para os casos maiores

```
BottomUp-Cut-Rod( $p, n$ )  
1  seja  $r[0 \dots n]$  um novo array  
2   $r[0] = 0$   
3  para  $j = 1$  até  $n$  faça  
4       $s = 0$   
5      para  $i = 1$  até  $j$  faça  
6           $t = p[i] + r[j - i]$   
7          se  $t > s$  então  
8               $s = t$   
9       $r[j] = s$   
10 devolva  $r[n]$ 
```

- ▶ o algoritmo tem complexidade de tempo $\Theta(n^2)$

Algoritmo de PD — Bottom-Up

- ▶ criamos um vetor auxiliar $r[0 \dots n]$ para usar programação dinâmica
- ▶ preenchemos do caso base para os casos maiores

```
BottomUp-Cut-Rod( $p, n$ )  
1  seja  $r[0 \dots n]$  um novo array  
2   $r[0] = 0$   
3  para  $j = 1$  até  $n$  faça  
4       $s = 0$   
5      para  $i = 1$  até  $j$  faça  
6           $t = p[i] + r[j - i]$   
7          se  $t > s$  então  
8               $s = t$   
9       $r[j] = s$   
10 devolva  $r[n]$ 
```

- ▶ o algoritmo tem complexidade de tempo $\Theta(n^2)$

Algoritmo de PD — Bottom-Up

- ▶ criamos um vetor auxiliar $r[0 \dots n]$ para usar programação dinâmica
- ▶ preenchemos do caso base para os casos maiores

```
BottomUp-Cut-Rod( $p, n$ )  
1  seja  $r[0 \dots n]$  um novo array  
2   $r[0] = 0$   
3  para  $j = 1$  até  $n$  faça  
4       $s = 0$   
5      para  $i = 1$  até  $j$  faça  
6           $t = p[i] + r[j - i]$   
7          se  $t > s$  então  
8               $s = t$   
9       $r[j] = s$   
10 devolva  $r[n]$ 
```

- ▶ o algoritmo tem complexidade de tempo $\Theta(n^2)$

Reconstruindo uma solução

- ▶ criamos um vetor $sol[1 \dots n]$. O slot $sol[j]$ guarda o tamanho i do primeiro pedaço a cortar quando estivermos resolvendo um subproblema de tamanho j .

Extended-BottomUp-Cut-Rod(p, n)

```
1  sejam  $r[0 \dots n]$  e  $sol[1 \dots n]$  novos arrays
2   $r[0] = 0$ 
3  para  $j = 1$  até  $n$  faça
4       $s = 0$ 
5      para  $i = 1$  até  $j$  faça
6           $t = p[i] + r[j - i]$ 
7          se  $t > s$  então
8               $s = t$ 
9               $sol[j] = i$ 
10      $r[j] = s$ 
11  devolva  $r$  e  $sol$ 
```

Reconstruindo uma solução

- ▶ criamos um vetor $sol[1 \dots n]$. O slot $sol[j]$ guarda o tamanho i do primeiro pedaço a cortar quando estivermos resolvendo um subproblema de tamanho j .

Extended-BottomUp-Cut-Rod(p, n)

```
1  sejam  $r[0 \dots n]$  e  $sol[1 \dots n]$  novos arrays
2   $r[0] = 0$ 
3  para  $j = 1$  até  $n$  faça
4       $s = 0$ 
5      para  $i = 1$  até  $j$  faça
6           $t = p[i] + r[j - i]$ 
7          se  $t > s$  então
8               $s = t$ 
9               $sol[j] = i$ 
10      $r[j] = s$ 
11  devolva  $r$  e  $sol$ 
```

Reconstruindo uma solução

Print-Cut-Rod-Solution(p, n)

```
1  ( $r, sol$ ) = EXTENDED-BOTTOMUP-CUT-ROD( $p, n$ )
2  enquanto  $n > 0$  faça
3      imprima  $sol[n]$ 
4       $n = n - sol[n]$ 
```

tamanho	0	1	2	3	4	5	6	7	8	9	10
p[i]	0	1	5	8	9	10	17	17	20	24	30
r[i]	0	1	5	8	10	13	17	18	22	25	30
sol[i]		1	2	3	2	2	6	1	2	3	10